Министерство образования Республики Беларусь
Учреждение образования
«Брестский государственный технический университет»
Кафедра ИИТ

Лабораторная работа №4
По дисциплине: «ОИвИС»
Тема: «Трекинг множественных объектов»

Выполнил:
Студент 4 курса
Группы ИИ-25
Пальчастый Я.М
Проверил:
Андренко К.В.

Брест 2026

Цель работы:
исследовать применение алгоритмов трекинга на базе обученной сети-детектора объектов

Общее задание

1. Используя сеть-детектор, обученный в ЛР 3, реализовать логику для отслеживания множественных объектов, используя библиотеку Ultralytics YOLO;
2. Применять алгоритмы BoT-Sort и ByteTrack (задействовать соответствующие конфигурационные файлы);
3. Исследовать изменения параметров в конфигурационных файлах и их влияние на качество трекинга;
4. В качестве исходных видеоматериалов для экспериментов использовать видео-ролики из сети (например, из YouTube), содержащие множественные объекты классов из ЛР 3;
5. Оформить отчет по выполненной работе, залить исходный код и отчет в соответствующий репозиторий на github.

Код программы:

```
import random
from pathlib import Path
import time, shutil, subprocess, yaml

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import cv2
import torch
from ultralytics import YOLO
from ultralytics.utils import ROOT
from google.colab import files
from ultralytics.trackers.basetrack import BaseTrack
from IPython.display import HTML, display
from base64 import b64encode

random.seed(42)
print("GPU доступен:", torch.cuda.is_available())


WEIGHTS = Path("runs/detect/runs_plates/yolo12n/weights/best.pt")
if not WEIGHTS.exists():
    print("best.pt не найден — выбери файл с компьютера")
    up = files.upload()
    WEIGHTS = Path(list(up.keys())[0])
print("Веса:", WEIGHTS)



VIDEO = Path("video.mp4")
up = files.upload()
if VIDEO.exists():          #
    VIDEO.unlink()
Path(list(up)[0]).rename(VIDEO)

cap = cv2.VideoCapture(str(VIDEO))
print("кадров:", int(cap.get(cv2.CAP_PROP_FRAME_COUNT)),
      "| fps:", cap.get(cv2.CAP_PROP_FPS),
      "| размер:", int(cap.get(3)), "x", int(cap.get(4)))
cap.release()
```

```
TRACKERS_DIR = ROOT / "cfg" / "trackers"
Path("trackers").mkdir(exist_ok=True)
for name in ["botsort.yaml", "bytetrack.yaml"]:
    shutil.copy(TRACKERS_DIR / name, Path("trackers") / name)

print(" bytetrack.yaml ")
print(Path("trackers/bytetrack.yaml").read_text())
print(" botsort.yaml ")
print(Path("trackers/botsort.yaml").read_text())


CONF = 0.25
PROJECT = "runs_track"


def run_tracking(tracker_cfg, label, folder, conf=CONF, save=True):
    BaseTrack.reset_id()
    model = YOLO(str(WEIGHTS))
    tracks = {}
    n_dets, n_frames = 0, 0

    t0 = time.time()
    for i, r in enumerate(model.track(
            source=str(VIDEO), tracker=tracker_cfg, conf=conf, imgsz=640,
            stream=True, save=save, project=PROJECT, name=folder,
            exist_ok=True, verbose=False)):
        n_frames += 1
        if r.boxes.id is not None:
            ids = r.boxes.id.int().tolist()
            n_dets += len(ids)
            for tid in ids:
                tracks.setdefault(tid, []).append(i)
    dt = time.time() - t0

    lens = np.array([len(v) for v in tracks.values()]) if tracks else
np.array([0])
    stats = {
        "эксперимент": label,
        "уникальных ID": len(tracks),
        "средняя длина трека (кадров)": round(float(lens.mean()), 1),
        "коротких треков (<15 кадров)": int((lens < 15).sum()),
        "объектов на кадр": round(n_dets / max(n_frames, 1), 2),
        "FPS": round(n_frames / dt, 1),
    }
    return stats, tracks


RESULTS, ALL_TRACKS = [], {}

BASE = [
    ("BoT-SORT (по умолчанию)",  "botsort_default",   "trackers/botsort.yaml"),
    ("ByteTrack (по умолчанию)", "bytetrack_default",
"trackers/bytetrack.yaml"),
]

for label, folder, cfg in BASE:
    stats, tracks = run_tracking(cfg, label, folder, save=True)
    RESULTS.append(stats)
    ALL_TRACKS[label] = tracks
    print("готово:", label)

display(pd.DataFrame(RESULTS))


def make_cfg(base_yaml, out_name, **changes):
    cfg = yaml.safe_load(Path(base_yaml).read_text())
    cfg.update(changes)
    out = Path("trackers") / out_name
```

```
    out.write_text(yaml.safe_dump(cfg, allow_unicode=True))
    return str(out)

BYTE, BOT = "trackers/bytetrack.yaml", "trackers/botsort.yaml"

EXPERIMENTS = [
    ("ByteTrack: track_buffer=10", "byte_buf10", BYTE, dict(track_buffer=10)),
    ("ByteTrack: track_buffer=90", "byte_buf90", BYTE, dict(track_buffer=90)),
    ("ByteTrack: track_high_thresh=0.5", "byte_high05", BYTE,
dict(track_high_thresh=0.5)),
    ("ByteTrack: match_thresh=0.5", "byte_match05", BYTE,
dict(match_thresh=0.5)),
    ("ByteTrack: match_thresh=0.95", "byte_match095", BYTE,
dict(match_thresh=0.95)),
    ("ByteTrack: conf детектора=0.1", "byte_conf01", BYTE, dict(conf=0.1)),
    ("BoT-SORT: track_buffer=10", "bot_buf10", BOT, dict(track_buffer=10)),
    ("BoT-SORT: track_buffer=90", "bot_buf90", BOT, dict(track_buffer=90)),
    ("BoT-SORT: gmc_method=none", "bot_nogmc", BOT, dict(gmc_method="none")),
    ("BoT-SORT: conf детектора=0.1", "bot_conf01", BOT, dict(conf=0.1)),
    ("BoT-SORT: with_reid=True", "bot_reid", BOT, dict(with_reid=True)),
]

for label, folder, base, changes in EXPERIMENTS:
    changes = dict(changes)
    conf = changes.pop("conf", CONF)
    cfg_path = make_cfg(base, f"{folder}.yaml", **changes)
    try:
        stats, tracks = run_tracking(cfg_path, label, folder, conf=conf,
save=False)
    except Exception as e:
        print(f"{label}: пропущено ({type(e).__name__}: {e})")
        continue
    RESULTS.append(stats)
    ALL_TRACKS[label] = tracks
    print("готово:", label)


df = pd.DataFrame(RESULTS)
cols = ["уникальных ID", "средняя длина трека (кадров)", "коротких треков (<15
кадров)"]

fig, axes = plt.subplots(1, 3, figsize=(18, 7), sharey=True)
for ax, col in zip(axes, cols):
    ax.barh(df["эксперимент"], df[col])
    ax.set_title(col)
    ax.grid(alpha=0.3, axis="x")
axes[0].invert_yaxis()
plt.tight_layout()
plt.show()

display(df)   # FIX


labels = ["BoT-SORT (по умолчанию)", "ByteTrack (по умолчанию)"]

fig, axes = plt.subplots(1, 2, figsize=(16, 7), sharex=True)
for ax, label in zip(axes, labels):
    items = sorted(ALL_TRACKS[label].items(), key=lambda kv: kv[1][0])
    for row, (tid, frames) in enumerate(items):
        ax.hlines(row, frames[0], frames[-1], linewidth=3)
    ax.set_title(f"{label}\nтреков: {len(items)}")
    ax.set_xlabel("номер кадра")
    ax.set_ylabel("трек (по порядку появления)")
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```

```
def show_tracked(folder):

    root = Path(PROJECT).resolve()
    found = list(root.rglob(f"{folder}/*.avi")) +
list(root.rglob(f"{folder}/*.mp4"))
    if not found:
        print("видео не найдено для:", folder, "| искали в:", root)
        return
    out = f"{folder}_h264.mp4"
    subprocess.run(["ffmpeg", "-y", "-loglevel", "error", "-i", str(found[0]),
                    "-vcodec", "libx264", "-pix_fmt", "yuv420p", "-crf", "28",
out])
    data = b64encode(open(out, "rb").read()).decode()
    display(HTML(f'<video height="480" controls><source
src="data:video/mp4; base64,{data}" type="video/mp4"></video>'))

show_tracked("bytetrack_default")
show_tracked("botsort_default")
```

```
GPU доступен: True
```

best.pt(n/a) - 5500954 bytes, last modified: n/a - 100% done
Saving best.pt to best.pt
Веса: best.pt

```
существует: True
размер: 5.5 МБ
```

video.MP4(video/mp4) - 2423483 bytes, last modified: n/a - 100% done
Saving video.MP4 to video.MP4
кадров: 213 | fps: 30.0 | размер: 406 x 720

bytetrack.yaml

```
# Ultralytics 🚀 AGPL-3.0 License - https://ultralytics.com/license

# ByteTrack tracker defaults for mode="track"
# Docs: https://docs.ultralytics.com/modes/track

tracker_type: bytetrack # (str) Tracker backend: bytetrack
track_high_thresh: 0.25 # (float) First-stage match threshold; raise for
cleaner tracks, lower to keep more
track_low_thresh: 0.1 # (float) Second-stage threshold for low-score matches;
balances recovery vs drift
```

new_track_thresh: 0.25 # (float) Start a new track if no match ≥ this; higher reduces false tracks
track_buffer: 30 # (int) Frames to keep lost tracks alive; higher handles occlusion, increases ID switches risk
match_thresh: 0.8 # (float) Association similarity threshold (IoU/cost); tune with detector quality
fuse_score: True # (bool) Fuse detection score with motion/IoU for matching; stabilizes weak detections

botsort.yaml

```
# Ultralytics 🚀 AGPL-3.0 License - https://ultralytics.com/license

# BoT-SORT tracker defaults for mode="track"
# Docs: https://docs.ultralytics.com/modes/track

tracker_type: botsort # (str) Tracker backend: botsort
track_high_thresh: 0.25 # (float) First-stage match threshold; raise for cleaner tracks, lower to keep more
track_low_thresh: 0.1 # (float) Second-stage threshold for low-score matches; balances recovery vs drift
new_track_thresh: 0.25 # (float) Start a new track if no match ≥ this; higher reduces false tracks
track_buffer: 30 # (int) Frames to keep lost tracks alive; higher handles occlusion, increases ID switches risk
match_thresh: 0.8 # (float) Association similarity threshold (IoU/cost); tune with detector quality
fuse_score: True # (bool) Fuse detection score with motion/IoU for matching; stabilizes weak detections

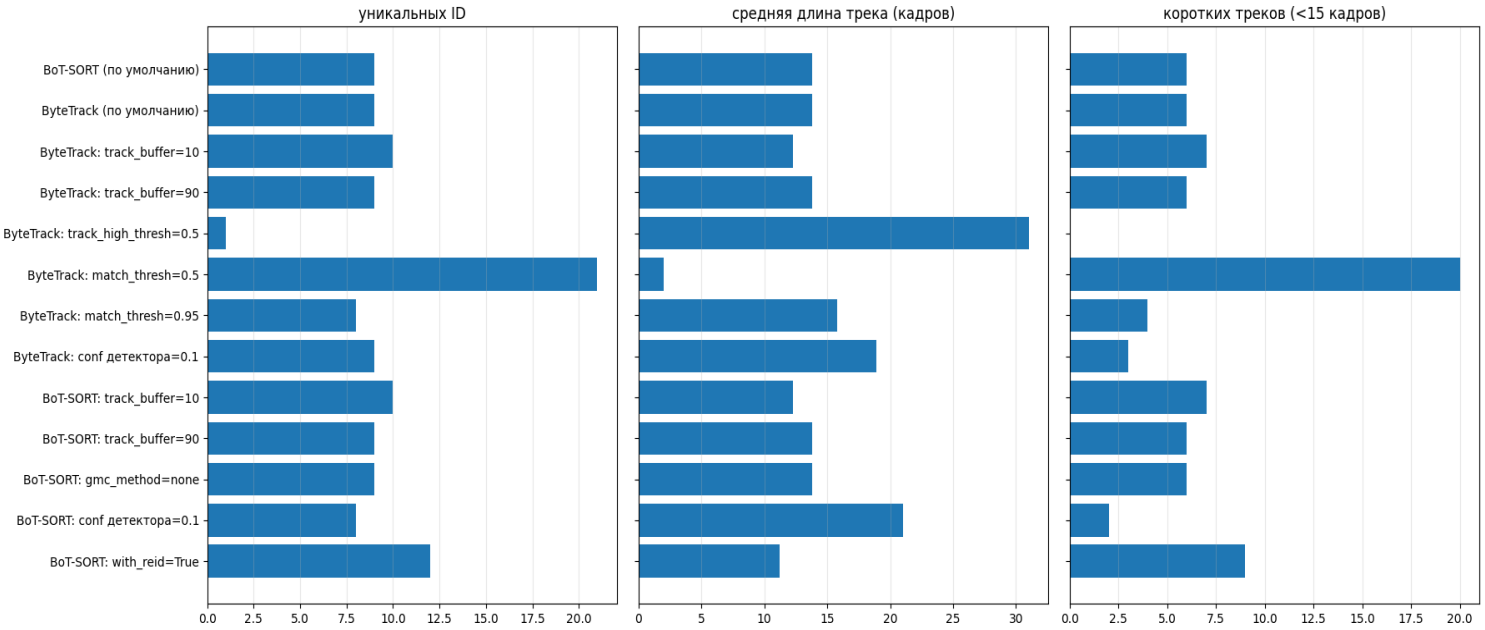
# BoT-SORT specifics
gmc_method: sparseOptFlow # (str) Global motion compensation: sparseOptFlow|orb|sift|ecc|none; helps moving camera scenes

# ReID model related thresh
proximity_thresh: 0.5 # (float) Min IoU to consider tracks proximate for ReID; higher is stricter
appearance_thresh: 0.8 # (float) Min appearance similarity for ReID; raise to avoid identity swaps
with_reid: False # (bool) Enable ReID model use; needs extra model and compute
model: auto # (str) ReID model name/path; "auto" uses detector features if available
```

Results saved to **/content/runs/detect/runs_track/botsort_default**
готово: BoT-SORT (по умолчанию)
Results saved to **/content/runs/detect/runs_track/bytetrack_default**
готово: ByteTrack (по умолчанию)

готово: ByteTrack: track_buffer=10
готово: ByteTrack: track_buffer=90
готово: ByteTrack: track_high_thresh=0.5
готово: ByteTrack: match_thresh=0.5

```
готово: ByteTrack: match_thresh=0.95 готово: ByteTrack:
conf детектора=0.1 готово: BoT-SORT: track_buffer=10
готово: BoT-SORT: track_buffer=90 готово: BoT-SORT:
gmc_method=none готово: BoT-SORT: conf детектора=0.1
готово: BoT-SORT: with_reid=True
```



эксперимент	уникальных ID	средняя длина трека (кадров)	коротких треков (<15 кадров)	объектов на кадр	FPS
BoT-SORT (по умолчанию)	9	13.8	6	0.58	12.0
ByteTrack (по умолчанию)	9	13.8	6	0.58	50.7
ByteTrack: track_buffer=10	10	12.3	7	0.58	40.7
ByteTrack: track_buffer=90	9	13.8	6	0.58	61.8
ByteTrack: track_high_thresh=0.5	1	31.0	0	0.15	63.1
ByteTrack: match_thresh=0.5	21	2.0	20	0.20	54.5
ByteTrack: match_thresh=0.95	8	15.8	4	0.59	56.3
ByteTrack: conf детектора=0.1	9	18.9	3	0.80	60.9
BoT-SORT: track_buffer=10	10	12.3	7	0.58	38.3
BoT-SORT: track_buffer=90	9	13.8	6	0.58	41.3

BoT-SORT: gmc_method=none	9	13.8	6	0.58	62.3
BoT-SORT: conf детектора=0.1	8	21.0	2	0.79	36.1
BoT-SORT: with_reid=True	12	11.2	9	0.63	41.0



Вывод: исследовал применение алгоритмов трекинга на базе обученной

сети-детектора объектов